GenSpark

Git & GitHub: Fundamentals, Branching Strategies, and Pull Requests

Last week you used Git and GitHub to complete your first commit and pull request. This week we go one level deeper: understanding what is actually happening under the hood, the branching strategies teams use to manage larger, ongoing projects, and how to handle the situations that every team runs into, such as merge conflicts, code reviews, and mistakes.

What Git Actually Tracks

Git is a version control system, but it is worth understanding what that means concretely. Git does not just save copies of your files. It records **snapshots** of your entire project at each commit, and it can show you the difference between any two snapshots, revert to a previous one, or merge changes from different snapshots together.

Three core concepts:

- **Repository (repo):** the full project, including its entire history of changes. A repo lives both locally on your machine and remotely on GitHub, and you sync between the two.
- **Commit:** a saved snapshot of your project at a point in time, along with a message describing what changed. Commits form a chain, each one building on the last. Every commit has a unique ID called a **hash**, shown in short form as seven characters, such as `38894b7` .
- **Branch:** an independent line of development. A branch lets you make changes without affecting the main codebase until you are ready to merge them back in. Technically a branch is just a movable label pointing at a commit, which is why creating one is instant.

Git also keeps track of **HEAD**, a pointer to where you are right now: the branch (and commit) you have checked out.

The three areas

A change moves through three places before it becomes history:

Area	What it is	In GitHub Desktop
Working folder	The files on your disk, where you edit	The files in PyCharm
Staging area	The set of changes you have chosen to include in the next commit	The **checkboxes** in the Changes tab
Repository	The saved commits	The History tab

This is why GitHub Desktop lists changed files with checkboxes. Checking a file *stages* it, and unchecking it keeps it out of the next commit. When you unchecked the `.idea` files in the Week 1 lab, you were choosing what to stage. Staging lets you edit ten files and commit only the three that belong together, so each commit tells one clear story.

GitHub Desktop and Git Commands

GitHub Desktop is a friendly layer over Git. Behind every button is a Git command, and on client projects you will often work from a terminal, in a Linux container, or in a pipeline, where there is no GitHub Desktop. Knowing both makes you flexible. You can run these in the PyCharm Terminal.

In GitHub Desktop	Git command
Clone repository	`git clone <url>`
New Branch	`git switch -c <name>`
Switch branch (Current Branch menu)	`git switch <name>`
Tick a file in Changes (stage it)	`git add <file>`
Commit	`git commit -m "message"`

Publish branch (the first push)	`git push -u origin <name>`
Push origin	`git push`
Fetch origin	`git fetch`
Pull origin	`git pull`
Branch > Update from main	`git merge main`
History tab	`git log`
Discard Changes	`git restore <file>`
Revert Changes in Commit	`git revert <hash>`
See what changed	`git status` and `git diff`

Three read-only commands are worth knowing right away:

bash

```
git status                  # what is changed, staged, or untracked
git diff                    # the exact lines you changed but have not s
git log --oneline           # one line per commit, newest first
```

A real example. After editing `settings.py`:

```
$ git status -s
 M settings.py

$ git diff
--- a/settings.py
+++ b/settings.py
@@ -1,2 +1,3 @@
 TIME_LIMIT = 15
 BOARD_SIZE = 3
+SHOW_HINTS = True
```

The `M` in the first column of `git status -s` means modified. The `+` line in the diff is an added line, and a `-` line would be a removed one. The same diff colors appear in GitHub Desktop and on a pull request page. Reading diffs is a core skill, because a pull request is a diff.

Why Branches Exist

Imagine two engineers both working on the same project at the same time. Without branches, they would be editing the same files simultaneously, constantly overwriting each other's work. Branches solve this by letting each person work in isolation. When `main` is a stable branch and each engineer works on their own branch, they only need to merge their changes back together when the work is actually ready. Git handles combining the changes, or flags a conflict if two people changed the same lines.

Branches are cheap, so use them freely. One branch per feature or fix is the usual rule.

Naming branches

A branch name should say what the work is. Common patterns use a short prefix and lowercase words joined by hyphens:

```
feature/add-login
feature/timed-moves
bugfix/date-parsing
hotfix/crash-on-empty-board
```

Some teams also include a ticket number, such as `feature/PROJ-142-timed-moves`. Follow the team's convention, and avoid vague names like `test` or `my-branch`.

Common Branching Strategies

Different teams use different conventions for how branches are structured. A few you will encounter:

Feature branching (what you used in Week 1): every new piece of work gets its own branch, named for what it does (`feature/add-login`, `bugfix/date-parsing`). When the work is ready, it is merged into `main` through a pull request. This is the most common approach for small to mid-sized teams and is what you will use throughout this program.

Git Flow: a more structured model with dedicated long-lived branches for different purposes. `main` holds production-ready code, `develop` holds work that is ready for the next release but not yet live, and short-lived feature, release, and hotfix branches merge into `develop` or `main` depending on their purpose. This adds more process overhead and is more common on larger, release-cycle-driven teams.

Trunk-based development: engineers commit small, frequent changes directly to `main` (or to very short-lived branches that merge quickly), relying heavily on automated testing to catch problems fast. This suits teams with mature CI/CD pipelines and a strong testing culture.

Strategy	Branches	Best for
Feature branching	`main` plus short-lived feature branches	Small to mid-sized teams, most client projects
Git Flow	`main`, `develop`, plus feature, release, and hotfix branches	Larger teams with scheduled releases
Trunk-based	`main` plus very short-lived branches	Teams with strong automated tests and CI/CD

You do not need to master all three. What matters is recognizing that "which branching strategy" is a real decision teams make, and that the feature-branch approach you are already using is a solid default for most situations, including most FDE client engagements.

GitHub: Where Repos Live and Collaboration Happens

Git is the tool. GitHub is a platform that hosts Git repositories remotely and adds collaboration features on top. Other platforms do the same job: GitLab and Bitbucket are the most common. The next article builds a pipeline on GitHub and shows the GitLab version for comparison.

- **Remote repository:** the copy of your repo hosted on GitHub, as opposed to the local copy on your machine. Git gives the remote a nickname, and for the repo you cloned it is called `origin`.
- **Clone:** download a remote repository to your machine, with its full history.
- **Fork:** a full copy of someone else's repository under your own account. It is useful when you do not have direct write access to the original and want to propose changes

independently. Forks are common in open source. On a client team, you are usually given access and use branches instead.

- **Default branch:** the main line of the repo. It is called `main` on new repos. Older repos often use `master`.
- **Pull request (PR):** a request to merge changes from one branch (or fork) into another, with a review step in between.

Push, fetch, and pull

Your local repo and the remote only talk when you tell them to:

Action	What it does	Changes your files?
Push	Sends your local commits up to the remote	No
Fetch	Downloads new commits from the remote, but does not apply them	No
Pull	Fetch, then merge the new commits into your current branch	Yes

Fetch is the safe look-before-you-leap step. After a teammate pushes a commit, a fetch tells you that you are behind but leaves your files alone:

```
$ git fetch
$ git status -sb
## main...origin/main [behind 1]
```

The `[behind 1]` means the remote has one commit you do not have. A `git pull` then brings it in. In GitHub Desktop, the top button shows which step is next: **Fetch origin** to check, **Pull origin** when there is something to download, and **Push origin** when you have commits to upload.

Before you start new work, make sure your `main` is up to date. Switch to `main`, fetch, and pull. Starting from an old copy is a common cause of conflicts.

Keeping Your Branch Up to Date

While you work on a branch, `main` keeps moving as teammates merge their changes. The longer you wait, the further your branch drifts. Pull the latest `main` into your branch regularly:

1. Make sure your own work is committed.
2. In GitHub Desktop, choose **Branch > Update from main**. (From the command line: `git switch main`, `git pull`, `git switch your-branch`, `git merge main`.)
3. Resolve any conflicts (see below), then keep working.

Doing this often keeps each update small and each conflict easy.

How Merging Works

When you merge a branch, Git combines two lines of history. There are two basic outcomes:

Fast-forward. If `main` has not moved since you branched, Git just slides the `main` label forward to your latest commit. No new commit is needed.

```
$ git merge feature/longer-timer
Updating 38894b7..b9bb56d
Fast-forward
 settings.py | 2 +-
```

Merge commit. If both branches have new commits, Git creates a new commit that has two parents and joins the histories. You saw one in the Week 1 lab: the entry "Merge pull request #1 from ..." in the History tab.

```
*   6d253f6 Merge feature/shorter-timer, keep 15 second limit
|\
| * 326cb4d Lower time limit to 10 seconds
* | b9bb56d Raise time limit to 20 seconds
|/
* 38894b7 Add hint setting
* 455fe14 Add game settings
```

You can draw this graph yourself with `git log --oneline --graph --all`. The two lines splitting and rejoining are the two branches.

Merge options on GitHub

On a pull request page, the green merge button has a drop-down with three ways to merge:

Option	What happens to the history	When teams like it
Create a merge commit	Keeps every commit from the branch and adds a merge commit	Default. Preserves the full story.
Squash and merge	Combines all the branch's commits into a single commit on `main`	Keeps `main` tidy when a branch has many small commits
Rebase and merge	Replays the branch's commits one by one on top of `main`, with no merge commit	A straight-line history

None is universally right. Teams pick one and stick to it, and a repository's settings can allow only some of them. The Week 1 lab used the default.

After merging, GitHub offers a **Delete branch** button. Use it. The commits are safe in `main`, and finished branches only clutter the list.

Handling Merge Conflicts

A merge conflict happens when Git cannot automatically combine two sets of changes because they touched the same lines of a file in different ways. This is not an error you did something wrong. It is an expected part of collaborative work.

Here is how one happens. `main` has `TIME_LIMIT = 15`. One branch changes it to 20 and gets merged. A second branch, started earlier, changes the same line to 10. Merging the second branch gives:

```
Auto-merging settings.py
CONFLICT (content): Merge conflict in settings.py
Automatic merge failed; fix conflicts and then commit the result.
```

Git has stopped and is waiting for you. The file now contains **conflict markers**:

```
<<<<<<< HEAD
TIME_LIMIT = 20
=======
TIME_LIMIT = 10
>>>>>>> feature/shorter-timer
BOARD_SIZE = 3
SHOW_HINTS = True
```

Read the markers like this:

- `<<<<<<< HEAD` begins your side, the branch you are merging *into*. The lines below it are your current version.
- `=======` divides the two versions.
- `>>>>>>> feature/shorter-timer` ends the other side, and names the branch being merged in.

Resolving a conflict

1. **Find the conflicted files.** GitHub Desktop lists them, and `git status` shows them as `both modified`.
2. **Decide what the file should say.** Keep one side, keep the other, or write a combination. This often needs a conversation: the right answer depends on what both people intended.
3. **Edit the file** so it contains the final text and **no markers**. Delete the `<<<<<<<`, `=======`, and `>>>>>>>` lines too. Leaving them in is a classic mistake that breaks your code.
4. **Mark it resolved and commit.** In GitHub Desktop, finish the merge once every conflict is resolved. From the command line: `git add settings.py`, then `git commit`.
5. **Run your tests.** A conflict that "merged cleanly" can still produce code that does not work.

Both GitHub Desktop and PyCharm help. GitHub Desktop opens the conflicted files in your editor. PyCharm has a three-panel merge tool that shows your version, the incoming version, and the result, with buttons to accept either side or edit by hand. Menu names vary by version.

If you get stuck, you can back out cleanly with `git merge --abort`, which returns everything to how it was before the merge.

Preventing conflicts

- **Keep branches small and short-lived.** The longer a branch diverges from `main` before merging, the more likely, and the more complex, conflicts become. This is one of the practical reasons small, frequent PRs are considered good practice.
- **Update from `main` often.**
- **Communicate.** If two people are about to edit the same file, say so.
- **Split code into small modules.** The more files, the less often two people touch the same lines. Your Tic-Tac-Toe project already does this.

Pull Requests in Depth

You already opened and merged a PR in Week 1. This week, look more closely at the review side of the tool.

What a PR page contains

- **Title and description.** What changed and why.
- **Conversation tab.** The discussion, reviews, and status checks.
- **Commits tab.** Each commit in the branch.
- **Files changed tab.** The full diff of what is being merged. This is where reviewers spend their time.
- **Checks.** Results from automated tests and pipelines, which you will build in the next article.

Writing a good PR

- **Keep it small and focused.** One purpose per PR. A 50 line change gets a careful review. A 2,000 line change gets a skim.
- **Write a useful title and description.** Say what changed, why, and how you tested it. Link to the requirement or ticket if there is one. Reviewers should not need to ask what the PR is for.
- **Review your own diff first.** Read the Files changed tab before asking anyone else. You will

spot leftover debug prints and stray files.

- **Make sure tests pass.** Run `pytest` before you push.
- **Do not mix unrelated changes.** A bug fix and a rename belong in separate PRs.
- **Use a draft PR for work in progress.** When you create the PR, choose the draft option to signal "not ready to merge, but I want early feedback."

Reviewing a PR

On the **Files changed** tab, you can:

- **Comment on a specific line.** Hover over a line and click the blue **+** button to attach a comment to that exact code.
- **Suggest a change.** Inside a comment, the suggestion button lets you propose the exact replacement text, and the author can accept it with one click.
- **Batch your comments.** Choose **Start a review** instead of single comments, so the author gets everything at once.
- **Finish the review.** Click **Review changes** and choose one of three outcomes:

Outcome	Meaning
Comment	General feedback, with no decision
Approve	The changes look good to merge
Request changes	Something must be fixed before this can merge

"Request changes" is different from a simple comment. It signals that the PR should not be merged until the problem is addressed. Use it for real problems (a bug, a missing test), and use plain comments for questions and optional ideas.

When you review, look at:

- Does it do what the description says?
- Are there tests, and do they cover the edge cases?

- Is the code readable, with clear names and no repeated code?
- Does it avoid unrelated changes?
- Are there any secrets or debug leftovers?

Be specific and kind. Comment on the code, not the person. "This loop skips the last item. Could we use `range(len(x))` here?" helps. "This is wrong" does not. When you are the author, answer every comment, push fixes to the same branch (the PR updates automatically), and say when something is done.

Protecting main

Teams usually protect `main` so nobody can push to it directly. Branch protection rules (called rulesets in newer GitHub settings) can require that every change comes through a pull request, that at least one person approves it, and that automated checks pass before the merge button works. This is how a team guarantees that `main` always contains reviewed, tested code. As an FDE, you may be the person who sets these up.

Different names, same idea

GitLab calls a pull request a **merge request** (MR). The workflow is the same: branch, push, open a request, review, merge. The next article builds a pipeline on GitHub and shows the GitLab version for comparison.

Undoing Things Safely

Everyone makes mistakes with Git. The right fix depends on how far the mistake has traveled:

Situation	What to do	GitHub Desktop
Edited a file and want the old version back	Discard the changes	Right-click the file in Changes, choose **Discard Changes**
Committed, but have **not pushed**	Undo the commit. Your changes return to the Changes list.	**Undo** button right after committing
Committed to the **wrong branch** (not pushed)	Undo, switch to the right branch, commit again	Undo, switch branch, commit

Already **pushed or merged**	Make a new commit that reverses it	History tab, right-click the commit, **Revert Changes in Commit**

Reverting adds a new commit instead of erasing the old one:

```
$ git revert --no-edit HEAD
[main 767cb35] Revert "Add board size"

$ git log --oneline
767cb35 Revert "Add board size"
246d9c6 Add board size
dfea103 Add settings
```

Rule of thumb: never rewrite history that other people have already seen. Commands like `git reset --hard` and `git push --force` can erase teammates' work, and they cannot be taken back easily. If you ever feel the need to force something, stop and ask a teammate first.

If you stash changes to switch branches, as GitHub Desktop offered in the Week 1 lab ("Leave my changes on ..."), the changes are tucked away safely on that branch. From the command line this is `git stash`.

What Never Belongs in Git

Git remembers everything, forever, so be careful what you give it.

Secrets. Never commit passwords, API keys, tokens, or `.env` files that contain them. Once pushed, treat a secret as **leaked**: other people can read the old commits even if you delete the file later. Remove it from the code, **rotate the key** (create a new one and cancel the old one), and tell your team. Store secrets in environment variables or a secrets manager. You will meet this again in the security week.

Generated and personal files. Things like `.venv/`, `.idea/`, `__pycache__/`, and `.DS_Store` do not belong in the repo. Anyone can rebuild them, and they differ from machine to machine.

Large files. Big data files and binaries bloat the repo for everyone, forever.

The tool for this is a `.gitignore` file in the project root: a list of file and folder patterns for Git to ignore.

```
.venv/
.idea/
__pycache__/
.env
```

When you chose the Python template while creating your repository, GitHub gave you a good starting list. One gotcha: `.gitignore` only affects files that Git is **not yet tracking**. If a file was already committed, adding it to `.gitignore` later does not remove it:

```
$ echo ".env" > .gitignore       # after .env was already committed
$ echo "SECRET_KEY = 'changed'" > .env
$ git status -s
 M .env                          # still tracked, still showing changes
```

An ignored file that was never committed stays out of the list, as it should. So write the `.gitignore` entry **before** the first commit.

Habits of Good Collaborators

- **Commit small and often,** each commit doing one thing.
- **Write good commit messages.** A short summary line that starts with a verb ("Add hint setting"), around 50 characters, and a longer description if needed. Say what changed and why, and avoid "fix" and "stuff". In GitHub Desktop, the message goes in the **Summary** box.
- **Pull before you start,** and update your branch regularly.
- **Do not commit to `main` directly** on a team project.
- **Delete merged branches.**
- **Read your diff before every commit.** The Changes tab shows exactly what will be saved.
- **Run the tests before you push.**

Common Mistakes

Mistake	What happens	Fix
Committing to `main` instead of a branch	Unreviewed changes land on `main`	Undo the commit, switch to a feature branch, commit again
Starting a branch from an old `main`	Extra conflicts later	Pull `main` first
Leaving conflict markers in a file	Broken code, or markers in your program	Remove every `<<<<<<<`, `=======`, and `>>>>>>>` line
Resolving a conflict without running the tests	A clean merge that does not work	Run `pytest` afterward
Huge branches that live for weeks	Painful conflicts and giant reviews	Merge small pieces often
Committing `.venv`, `.idea`, or `.env`	Clutter, or leaked secrets	Add them to `.gitignore` before the first commit
Adding a file to `.gitignore` after committing it	It stays tracked	Remove it from tracking and ignore it, or avoid the mistake
Pushing a secret, then deleting the file	The secret is still in the history	Rotate the secret immediately
Using `git push --force` on shared branches	Teammates' work disappears	Do not rewrite shared history
Mixing unrelated changes in one PR	Hard to review	One purpose per PR
A vague PR description	Reviewers must guess	Say what, why, and how it was tested
Merging your own PR without review on a team project	Mistakes reach `main`	Ask for a review
Confusing fetch and pull	Surprised by files changing, or by not seeing updates	Fetch to look, pull to update
Writing the commit message in the Description box	The commit title is the default text	Use the Summary box

Try It Yourself

Practice in a throwaway repository, not your real project. Create a new GitHub repo with a README and clone it with GitHub Desktop.

1. Open the PyCharm Terminal in the repo and run `git status`, `git log --oneline`, and `git branch`. Explain each output in a sentence.
2. Create a file, edit it, and run `git status -s` and `git diff` at each step. Stage the change by ticking it in GitHub Desktop, run `git status -s` again, and note the difference.
3. Make three small commits on a feature branch. Run `git log --oneline` and read the history.
4. Create a branch called `feature/hello`, push it, open a PR, and merge it using **Create a merge commit**. Run `git log --oneline --graph --all`.
5. Repeat exercise 4, but use **Squash and merge**. How does the history differ?
6. Have a second copy of the repo (clone it to another folder to play the part of a teammate). Push a commit from one copy, then use **Fetch origin** in the other. What does GitHub Desktop show? Then pull.
7. Create a deliberate conflict: edit the same line of one file on two branches, merge one branch into `main`, then try to merge the other. Find the markers, resolve the conflict, and commit. Then do it again and use `git merge --abort`.
8. Start a branch, let `main` move on, then use **Branch > Update from main** to bring your branch up to date.
9. Make a commit on `main` by mistake and use **Undo**. Then make another one, push it, and use **Revert Changes in Commit**. What is the difference in the history?
10. Add a `.env` file with a fake secret, and write the `.gitignore` entry **before** you commit. Confirm Git ignores it. Then try the opposite order and see what happens.
11. Open a PR with a deliberate small mistake. Ask a classmate to review it: one inline comment, one suggestion, and a **Request changes** review. Fix it and get an approval.
12. Write a PR description using the "what, why, how tested" pattern for your Tic-Tac-Toe

project.

13. List three branch names for tasks in your Tic-Tac-Toe plan that follow a clear convention.

14. Look at the Network or Insights graph of a public open source repository on GitHub. How do the contributors use branches and forks?

Why This Matters for an FDE

On a client engagement, you may be the one setting up repository structure and branching conventions for the first time, not just following an existing one. Understanding why different branching strategies exist, not just how to run the commands, means you can make a sensible recommendation based on the team's size and release cadence, rather than defaulting to whatever you personally used last. You will also often work in places with no friendly desktop app: a server, a container, a pipeline. Knowing the commands behind the buttons, how to read a diff, and how to recover from a conflict or a mistake is what makes you effective there. And habits such as small PRs, careful review, and never committing a secret are what clients notice when they decide whether to trust you with their code.

Comprehension Check

1. What is the difference between a commit and a branch?

2. What are the three areas a change moves through in Git, and which part of GitHub Desktop corresponds to staging?

3. What is the difference between `git fetch` and `git pull`? What does `[behind 1]` mean?

4. Why do small, frequent merges tend to cause fewer conflicts than large, infrequent ones?

5. What do the lines `<<<<<<<`, `=======`, and `>>>>>>>` mean in a conflicted file, and what must you do with them?

6. Name two ways to get out of a bad situation during a merge conflict.

7. What is the difference between **Create a merge commit** and **Squash and merge**?

8. If you were setting up a new repo for a two-person client project with frequent small releases, which branching strategy would you recommend, and why?

9. What is the difference between **Comment** and **Request changes** in a PR review?
10. You committed a change by mistake and have not pushed it. Then a different mistake was already pushed. How do you handle each?
11. You accidentally pushed an API key. Deleting the file and pushing again is not enough. Why, and what must you do?
12. A file is already tracked by Git, and you add it to `.gitignore`. Why does it still show changes?

← Previous: SDLC Foundations: Requirements Gathering and System Design Basics

Next: CI/CD Pipelines with GitHub Actions: Concepts and Building Pipeline Jobs →

0 comments so far

6

Add a comment...

Comment cannot be empty

Comment

No comments yet. Be the first to start the discussion!